

IT 161 – INFORMATION MANAGEMENT (PHASE I CLASS PROJECT)

National Disaster Relief & Resource Distribution Management System (NDRDMS)

Complete Operational Information System Development & Technical Defense Package

COURSE & TERM: IT 161 – Information Management (AY 2024–2025)	DATABASE PLATFORM: PostgreSQL 15 on Supabase Cloud	APPLICATION TECH: Java 23 / 24, JDBC 42.7.3, Swing GUI
ARCHITECTURE: 3-Tier Layered Architecture (3NF Schema)	TABLES IMPLEMENTED: 13 Normalized Relational Tables	COMPLIANCE RATING: 100% Meets All Rubric Requirements

1. Executive Summary & Case Organization

The **National Disaster Relief & Resource Distribution Management System (NDRDMS)** is an enterprise-grade operational information system designed to coordinate disaster mitigation, donor relief tracking, logistics dispatching, and community aid allocation during national emergency crises.

The system addresses common operational challenges faced by emergency command agencies (such as NDRRMC, DSWD, and Philippine Red Cross):

- **Siloed Relief Operations:** Consolidates disaster incident monitoring, donor contributions, and NGO aid requests under a unified, relational database backend.
- **Split-Transaction Tracking:** Normalizes individual contributions and request item lines into 3NF parent/child tables to eliminate multi-valued dependencies.
- **Role-Based Operational Security:** Enforces horizontal and vertical data isolation so Donors, Partner Organizations, Field Volunteers, and Command Center Administrators access dedicated, authenticated portals.

2. Normalized Relational Database Design (3NF)

The database architecture consists of **13 related tables** mapped with primary keys, foreign keys, cascade deletes, and check constraints to preserve referential integrity:

#	TABLE NAME	PRIMARY KEY	FOREIGN KEYS & REFERENCES	DESCRIPTION & 3NF ROLE
1	UserAccount	user_id (PK)	None	Authentication master store; SHA-256 password hash; Role CHECK constraint.
2	Donor	donor_id (PK)	user_id → UserAccount	1:1 subtype table for donors (Individual or Corporate classification).
3	Organization	org_id (PK)	user_id → UserAccount	1:1 subtype table for partner NGOs, LGUs, and humanitarian agencies.
4	Volunteer	volunteer_id (PK)	user_id → UserAccount	1:1 subtype table for field dispatchers, vehicle mode, and availability.
5	DisasterEvent	event_id (PK)	None	Master incident registry (Typhoons, Earthquakes, Volcanic unrest).
6	ReliefGood	good_id (PK)	None	Catalog of relief items (Water, Food, Medicine, Non-Food, Care).
7	Inventory	inventory_id (PK)	good_id → ReliefGood	Real-time warehouse tracking for current in-stock quantities.
8	Donation	donation_id (PK)	donor_id → Donor , event_id → DisasterEvent	Transaction header for donated batches and targeted disaster event.
9	DonationItem	item_id (PK)	donation_id → Donation , good_id → ReliefGood	Transaction line items resolving 3NF multi-item donation batches.
10	ReliefRequest	request_id (PK)	org_id → Organization , event_id → DisasterEvent	Aid request header submitted by community organizations and LGUs.
11	RequestItem	item_id (PK)	request_id → ReliefRequest , good_id → ReliefGood	Transaction line items resolving 3NF requested relief good quantities.
12	Delivery	delivery_id (PK)	request_id → ReliefRequest , volunteer_id → Volunteer	Logistics dispatch linking approved aid requests to volunteer fleet drivers.
13	Notification	notification_id (PK)	user_id → UserAccount	Audit log and event notification message trail for all users.

3. Live Database Verification (Supabase PostgreSQL)

The schema and seed data have been migrated and verified live in the cloud database. Every major table exceeds the minimum requirement of 10 sample records:

TABLE NAME	VERIFIED LIVE ROW COUNT	PHASE 1 RUBRIC STATUS
UserAccount	31 rows	Exceeds Requirement (≥ 10)
Donor	10 rows	100% Compliant
Organization	10 rows	100% Compliant
Volunteer	10 rows	100% Compliant
DisasterEvent	10 rows	100% Compliant
ReliefGood	10 rows	100% Compliant
Inventory	10 rows	100% Compliant
Donation	12 rows	100% Compliant
DonationItem	15 rows	100% Compliant
ReliefRequest	12 rows	100% Compliant
RequestItem	15 rows	100% Compliant
Delivery	10 rows	100% Compliant
Notification	10 rows	100% Compliant

4. JDBC Implementation & CRUD Architecture

All backend database communication is executed through **PostgreSQL JDBC 42.7.3** with parameterized queries:

Core CRUD Capabilities Implemented:

- **Create (C):**
 - Atomic split transactions inserting donation headers (RETURNING donation_id) followed by child line items.
 - Relief request creation inserting request headers followed by batch item lines.
 - New disaster event registration and catalog inventory items creation.
- **Read (R):**
 - Full relational data tables rendered across all 12 modules.
 - Role-isolated historical tracking (WHERE user_id = ?) ensuring privacy for donors, organizations, and volunteers.
- **Update (U):**

- Updating donation statuses (Approved, Pending).
- Updating relief request approval status.
- Updating disaster event states (Active ↔ Resolved).
- Updating fleet delivery logistics (In Transit → Delivered).
- Updating warehouse stock counts and volunteer dispatch availability.
- **Delete (D):**
 - Permanent SQL record removal with confirmation warnings on Donations, Relief Requests, Disaster Events, Inventory Goods, and Deliveries.
- **Search Functionality:**
 - Real-time table filtering in the user interface.
 - Backend SQL ILIKE ? query methods implemented across all major DAOs.

5. Operational Reports Requirement

The system implements **three (3) multi-table relational summary reports** accessible via the **Reports** module:

Report 1: Disaster Relief Operational Summary Report

Aggregates active and resolved disaster incidents with live operational metrics using relational joins:

```
SELECT
    e.event_id, e.event_name, e.location, e.status,
    COUNT(DISTINCT d.donation_id) AS total_donations,
    COUNT(DISTINCT r.request_id) AS total_requests,
    COUNT(DISTINCT del.delivery_id) AS total_deliveries
FROM DisasterEvent e
LEFT JOIN Donation d ON e.event_id = d.event_id
LEFT JOIN ReliefRequest r ON e.event_id = r.event_id
LEFT JOIN Delivery del ON r.request_id = del.request_id
GROUP BY e.event_id, e.event_name, e.location, e.status
ORDER BY e.event_id;
```

Report 2: Warehouse Inventory Stock Status & Replenishment Alert Report

Compares inventory stock counts against cumulative donations and requests with automated threshold status:

```
SELECT
    g.good_id, g.item_name, g.category, g.unit_of_measure,
    COALESCE(i.current_stock, 0) AS current_stock,
    COALESCE(SUM(di.quantity), 0) AS total_donated_qty,
    COALESCE(SUM(ri.quantity), 0) AS total_requested_qty,
    CASE
        WHEN COALESCE(i.current_stock, 0) <= 200 THEN 'CRITICAL ALERT'
        WHEN COALESCE(i.current_stock, 0) <= 1000 THEN 'LOW STOCK'
        ELSE 'SUFFICIENT'
    END AS stock_health
FROM ReliefGood g
LEFT JOIN Inventory i ON g.good_id = i.good_id
LEFT JOIN DonationItem di ON g.good_id = di.good_id
LEFT JOIN RequestItem ri ON g.good_id = ri.good_id
GROUP BY g.good_id, g.item_name, g.category, g.unit_of_measure, i.current_stock
ORDER BY i.current_stock ASC;
```

Report 3: Donor Contribution & Impact Audit Ledger

Analyzes donor participation, classification types, events supported, and contributed volumes:

```
SELECT
    dn.donor_id, u.full_name AS donor_name, dn.donor_type, u.contact_info,
    COUNT(DISTINCT d.donation_id) AS total_donations,
    COUNT(DISTINCT d.event_id) AS events_supported,
    COALESCE(SUM(di.quantity), 0) AS total_items_donated
FROM Donor dn
JOIN UserAccount u ON dn.user_id = u.user_id
LEFT JOIN Donation d ON dn.donor_id = d.donor_id
```

```
LEFT JOIN DonationItem di ON d.donation_id = di.donation_id  
GROUP BY dn.donor_id, u.full_name, dn.donor_type, u.contact_info  
ORDER BY total_donations DESC;
```

6. Presentation Demonstration Walkthrough (10–15 Minutes)

Follow this sequence during your oral defense to highlight your backend database compliance:

TIME	SEGMENT	WHAT TO SHOW & SAY	BACKEND CONCEPT HIGHLIGHTED
0:00 - 2:00	Introduction & ERD	State the project title, introduce team members, and briefly describe how the 13-table schema solves disaster coordination in 3NF.	Database Normalization, Referential Integrity.
2:00 - 5:00	Citizen / Donor Portal	1. Log in as donor1 / pass123. 2. Point out the Public Transparency Feed. 3. Go to Make a Donation: Add Bottled Water and Rice, then submit. 4. Go to My Contributions to verify that only donor1's records appear.	Split-Transaction SQL Insert (RETURNING clause), Role-Based Row Filtering (WHERE user_id = ?).
5:00 - 9:00	Admin Operations & Full CRUD	1. Log in as admin / admin123. 2. Show the Executive Dashboard with live PostgreSQL COUNT(*) analytics. 3. Go to Donations: Select the newly submitted donation, update status to Approved, click Save Changes. 4. Go to Disaster Events: Click + New Event, create an event, then demonstrate Toggle Status and Delete.	JDBC SQL UPDATE, SQL DELETE with cascade integrity, PreparedStatement.
9:00 - 12:00	Operational Reports & Analytics	1. Open the Reports tab in Admin Portal. 2. Switch between all 3 operational reports using the dropdown. 3. Explain to the instructor that these are live SQL aggregate queries using JOIN, GROUP BY, and COUNT(DISTINCT).	SQL Aggregate Functions, Conditional Expressions (CASE WHEN), Multi-table JOINS.
12:00 - 15:00	Field Volunteer & Conclusion	1. Log in as vol1 / pass123. 2. Go to Profile: Update contact number and toggle deployment availability, then click Save Profile. 3. Conclude by answering instructor questions.	SQL UPSERT / UPDATE, Input Validation with FormUtils.

7. Defense Q&A: Common Instructor Questions & Answers

Q1: Why did your group normalize the database into 13 tables?

Answer: "To adhere strictly to Third Normal Form (3NF). Transactions like Donations and Aid Requests can include multiple relief items. Creating separate line-item tables (DonationItem and RequestItem) removes multi-valued dependencies and eliminates repeating groups, ensuring data integrity."

Q2: How does your Java application prevent SQL injection?

Answer: "Every database operation in our Data Access Objects (DAOs) strictly utilizes PreparedStatement with

parameterized placeholders (?). We never concatenate raw user input into SQL query strings."

Q3: How do you handle password security?

Answer: "Passwords are not stored in plaintext. In `PasswordUtils.java`, all passwords are automatically digested using SHA-256 and stored as Base64-encoded hashes. During authentication, the input is hashed and compared against the stored hash."

Q4: How did you implement your operational reports?

Answer: "We created `ReportDAO.java`, which executes relational SQL queries utilizing `LEFT JOIN`, `GROUP BY`, and aggregate functions like `COUNT(DISTINCT)` and `SUM()` to compute metrics directly on PostgreSQL before returning the result sets to Java."

8. System Test Credentials Reference

ROLE	USERNAME	PASSWORD	TARGET PORTAL	PRIVILEGES / PURPOSE
Administrator	admin	admin123	Admin Operations Portal	Full system CRUD access, user management, and operational reporting.
Donor	donor1	pass123	Citizen & Donor Portal	Submits donations, tracks personal contributions, views public feed.
Volunteer	vol1	pass123	Field Volunteer Portal	Views assigned deliveries, updates availability and emergency contact info.
Organization	org1	pass123	Organization Portal	Submits community aid requests and tracks request fulfillment status.